VLSI Design

Verilog Practice Sheet

Varun Shastry (2023112005)

October 6, 2024

Exercise 9 Sequential Circuits

a) Given the set of State Equations

In this question, we are given a sequential circuit with 2 D-flip-flops, namely A and B, with two inputs X and Y, with Z being the only output.

The boolean state equations are given as:

$$A(t+1) = X \cdot Y' + X \cdot B$$

$$B(t+1) = X \cdot A + X \cdot B'$$

$$Z = A$$

Given these equations, we can infer that the output is solely a function of the present-state(s) of the flip-flops, and not of the inputs `X` or `Y`. Thus, this is a **Moore Machine**. We can draw the circuit diagram as below:

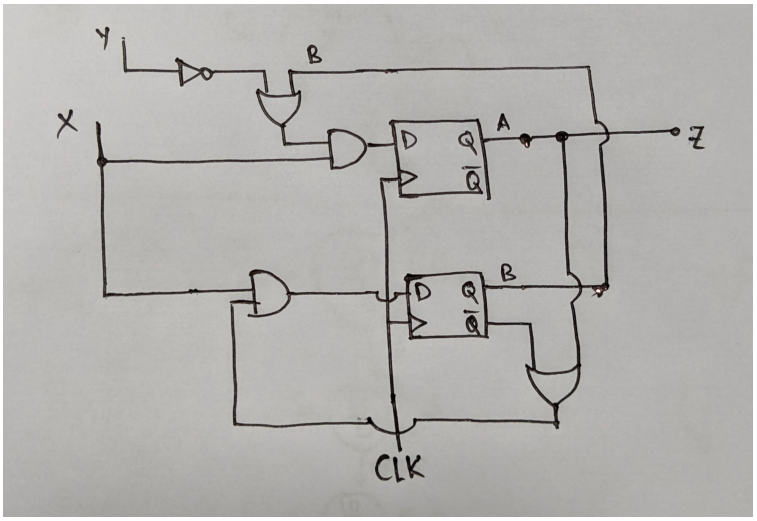


Figure 1: Circuit Diagram

Also, the state table can be made as below:

Current State		Input		Next State		Output
A	**B**	**X**	**Y**	**A(t+1)**	**B(t+1)**	**Z**
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	1	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	1	1
1	0	1	1	0	1	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

The state-transition diagram is given below:

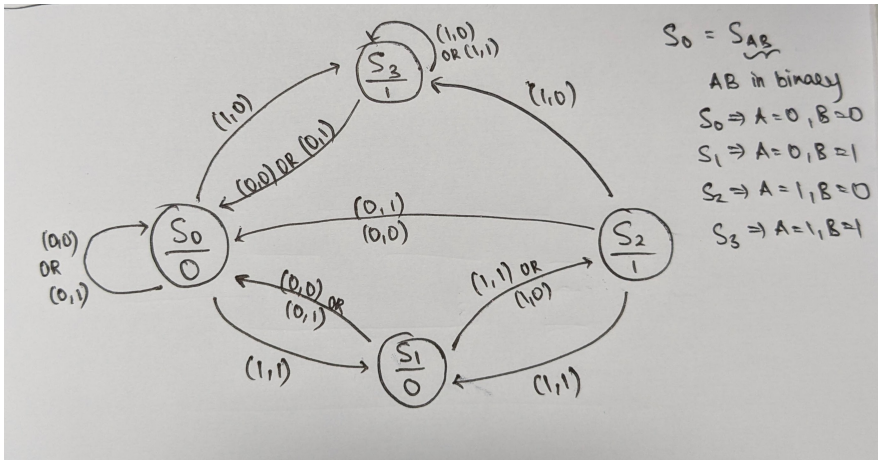


Figure 2: State Transition Diagram

We could implement this circuit in Verilog HDL, either through the structural method using the state equations given to us, or the behavioural, using the states and conditionals. I have implemented this circuit by writing the structural verilog code, which is given below:

```verilog
// Exercise 9 (a) from Verilog Practice Sheet

// D-flipflop (posiitve edge triggered)
module D_flipflop(D, CLK, Q);
    input D, CLK;
    output reg Q;

    initial begin
        Q = 1'b0;
    end

    always @(posedge CLK)
    begin
        Q <= D;
    end
endmodule

module Circuit(X, Y, CLK, Z, A, B);
    input X, Y, CLK;
    output Z, A, B;

    wire A_next, B_next;

    assign A_next = X & ((~Y) | B);
    assign B_next = X & ((~B) | A);

    D_flipflop u1 (A_next, CLK, A);
    D_flipflop u2 (B_next, CLK, B);

    assign Z = A;
endmodule

```

Figure 3: Verilog Module

I have used the D-flip-flop code, as done in Exercise 5, such that it is positive edge triggered. The D-flip-flop is transparent in the sense that the output is directly transferred from the input, at the arrival of the active clock edge, (in this case, the rising edge).

The module for the Circuit is simple to understand. The inputs for the two D-latches (corresponding to A and B respectively) are called `A_next` and `B_next`. These parameters are defined as wires, and are defined just as the equations given to us in the problem statement.
Finally, we assign the output variable `Z` to be A, after the instantiation of the D-flipflops.

NOTE that I have not implemented a `clear` input to reset the flipflops to a zero condition, as it is not specifically required for this problem, although implementing it is not too tough, it can be simply added as an input to the D-flipflop and then the condition for it could be written in the `always` block.

The testbench for this module is given below:

```
// Testbench for Exercise 9(a) from Verilog Practice Sheet
`timescale 1ns/1ns
`include "Exercise9_a.v"

module Exercise9_atb;
    reg X, Y, CLK;
    wire Z, A, B;

    Circuit uut (X, Y, CLK, Z, A, B);
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("Exercise9_atb.vcd");
        $dumpvars(0, Exercise9_atb);

        X = 0; Y = 0;
        #20;

        for (integer i = 0; i < 4; i = i + 1) begin
            {X, Y} = i;
            #20;
        end
        $finish;
    end

    initial begin
        $monitor("Time = %0t, X = %b, Y = %b, A = %b, B = %b, Z = %b", $time, X, Y, A, B, Z);
    end
endmodule
```

Figure 4: Testbench for above Module

The clock is flipped every 5ns, and thus the frequency of this clock is a 100MHz. The con of writing the structural model is that we cannot possibly cycle through all the states, since we force a certain input, in this case {`X=0, Y=0`}, and then iterate over all possible combinations of X and Y, but not all possible combinations of A, B, X and Y (from the state table above). This HDL is still valid, and can be tested for multiple rows of the above table, by tweaking the reset values of A and B to be values other than 0 and 0, which is implemented in the module of the D-flipflop.

Upon the execution of this file, we get the GTKWave output to be the image below:

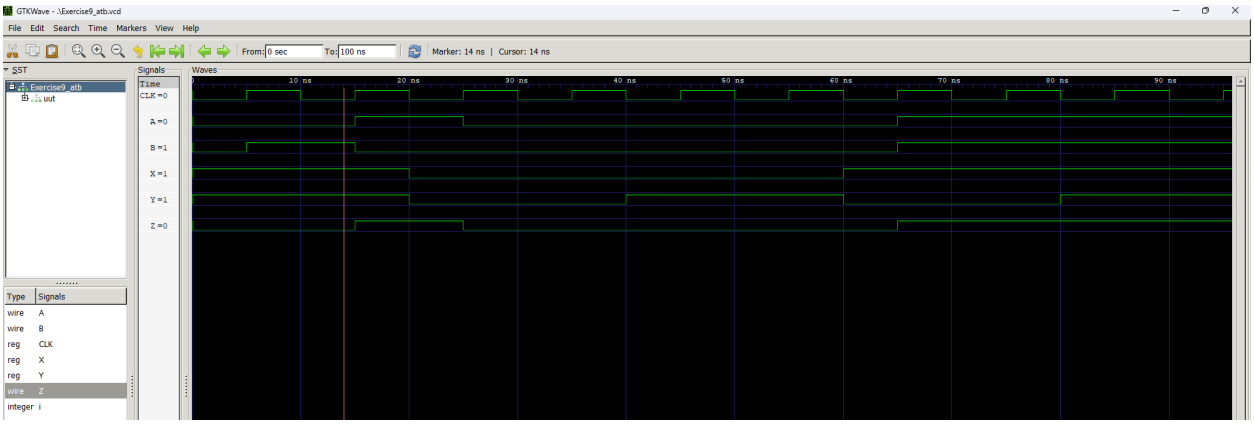


Figure 5: Waveforms seen in GTKWave

Let's try to verify the output of this circuit here at the red cursor above. We can see that currently, we have A = 0, B = 1, X = 1, Y = 1, Z = 0. From the state table above we can verify that the output on the arrival of the next positive clock edge is going to be: A(t+1) = 1, B(t+1) = 0, and thus Z = 1. Which can be seen in the image above. This can be similarly verified for the other points of the rising clock edge too.

b) Writing State Diagram and Verilog Code for given Circuit

We are given below, the circuit diagram of a sequential circuit:

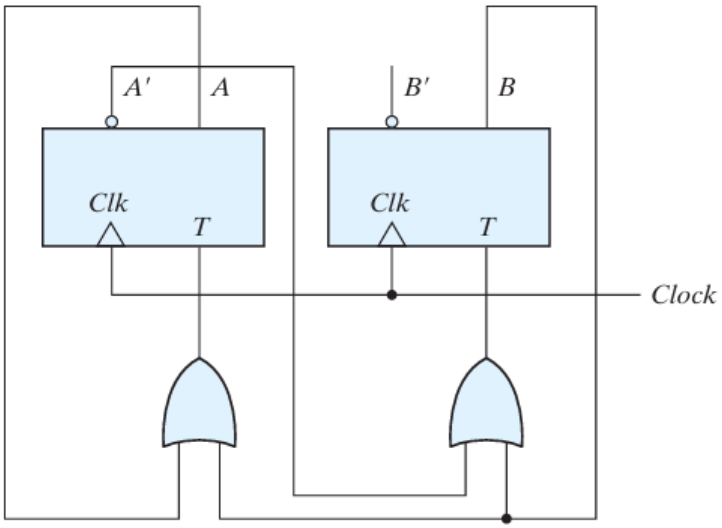


Figure 6: Circuit Diagram

This sequential circuit is self-sustained in the sense there are no external-inputs, the circuit is driven by it's own initial conditions set at the time of it's assembly. We have 2 T-flip-flops given, and thus the total maximum number of possible states, is $2^2 = 4$.

The T-flip-flop is another basic flip-flop, (the 'T' stands for toggle), and can be implemented using a D-flip-flop such that the output of the D-flipflop can be XOR'd with the input (T), and fed as an input to the D-flipflop. Thus, with the arrival of the active clock-edge, if the input T is 0, then the T-flipflop stores the output, but if T = 1, the output toggles. The state table for the T-flipflop is given below:

Clock	Current State (Q)	T	Next State (Q')	Comment
1	0	0	0	No Change
1	0	1	1	Toggle
1	1	0	1	No Change
1	1	1	0	Toggle
0	0	X	0	No Change
	1	X	1	No Change

With this understanding of the T-flipflop, we can write the logic for this circuit as below:

$$A(t+1) = \begin{cases} A', & \text{if } A(t) + B(t) = 1 \\ A, & \text{if } A(t) + B(t) = 0 \end{cases}$$

$$B(t+1) = \begin{cases} B', & \text{if } A'(t) + B(t) = 1 \\ B, & \text{if } A'(t) + B(t) = 0 \end{cases}$$

Thus the state table can be given as below:

Current State		**Next State**	
A	**B**	**A(t+1)**	**B(t+1)**
0	0	0	1
0	1	1	0
1	0	0	0
1	1	0	0

And, the state-transition diagram is given below:

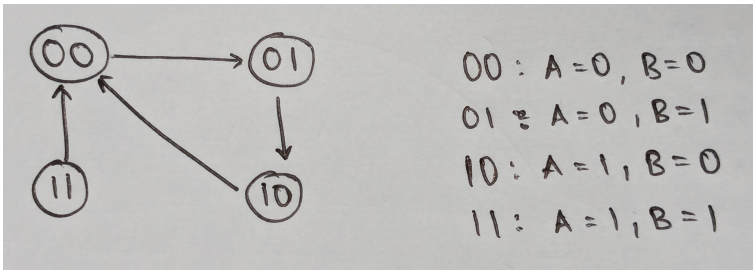
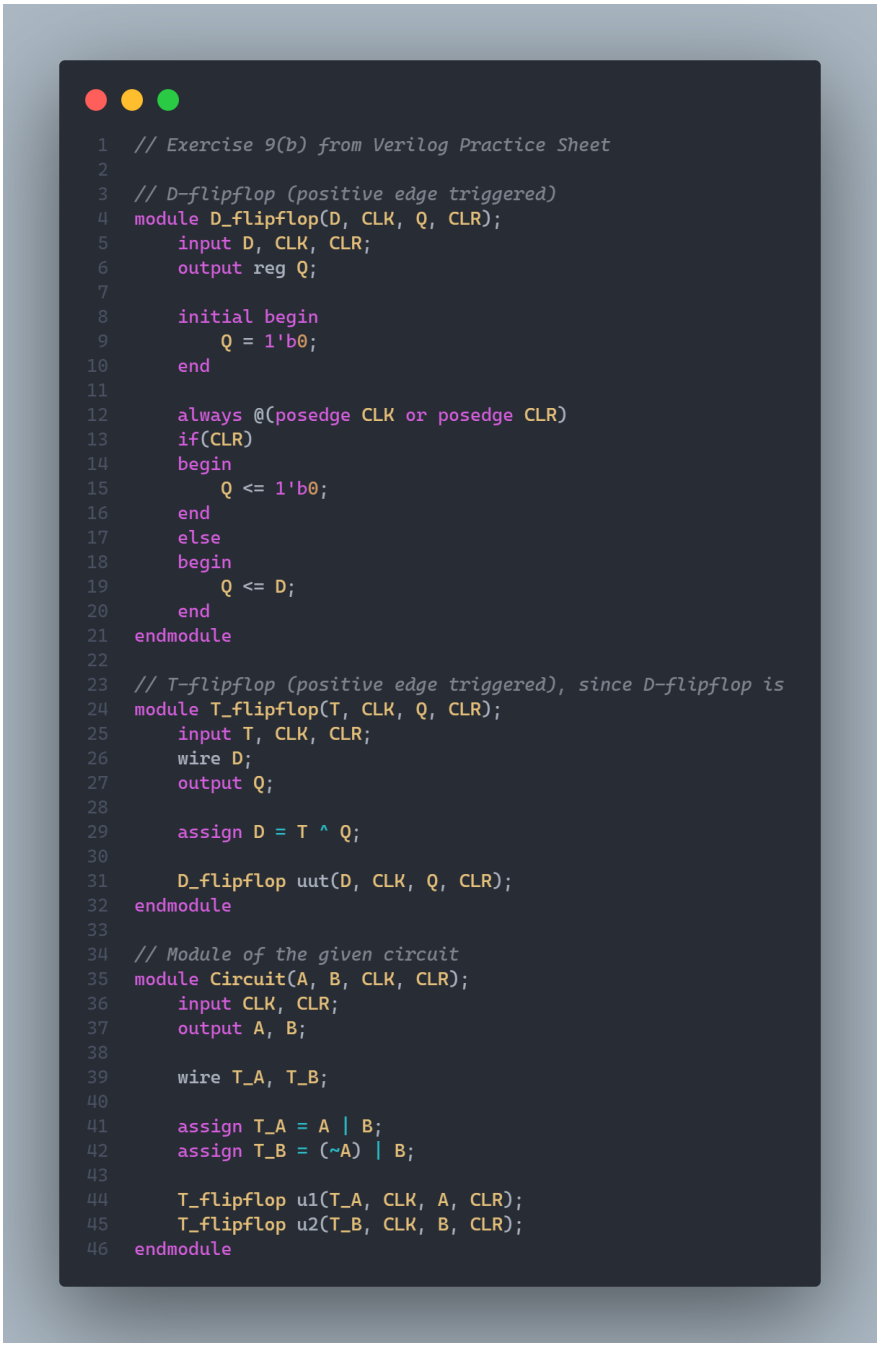


Figure 7: State transition Diagram

Thus this circuit acts like a counter, in a way that, once it starts from 3, it never reaches it again, so the counter is not circular in fashion! Instead, it counts from 0 to 2, and back to 0, in all cases except where we start from 3, wherein it counts from 3, to 0, to 2, and then back to 0.

The Modules defined for this code in Verilog HDL are given below:



```
// Exercise 9(b) from Verilog Practice Sheet

// D-flipflop (positive edge triggered)
module D_flipflop(D, CLK, Q, CLR);
    input D, CLK, CLR;
    output reg Q;

    initial begin
        Q = 1'b0;
    end

    always @(posedge CLK or posedge CLR)
    if(CLR)
    begin
        Q <= 1'b0;
    end
    else
    begin
        Q <= D;
    end
endmodule

// T-flipflop (positive edge triggered), since D-flipflop is
module T_flipflop(T, CLK, Q, CLR);
    input T, CLK, CLR;
    wire D;
    output Q;

    assign D = T ^ Q;

    D_flipflop uut(D, CLK, Q, CLR);
endmodule

// Module of the given circuit
module Circuit(A, B, CLK, CLR);
    input CLK, CLR;
    output A, B;

    wire T_A, T_B;

    assign T_A = A | B;
    assign T_B = (~A) | B;

    T_flipflop u1(T_A, CLK, A, CLR);
    T_flipflop u2(T_B, CLK, B, CLR);
endmodule
```

Figure 8: Module File

We use the same D-flipflop as mentioned above, with the addition of the CLR (clear) input, which is used to clear the sequential circuit when we start counting in it. This module is in-turn used for instantiating the T-flip-flop in the final `Circuit` module. The assignments in this module are pretty easy to understand, as they have been made by observation of the circuit diagram given to us.

The test-bench for this module file is given below:

```verilog
// Testbench for Exercise 9(b) from Verilog Practice Sheet
`timescale 1ns/1ns
`include "Exercise9_b.v"

module Exercise9_tb;
    reg CLK, CLR;
    wire A, B;

    Circuit uut(A, B, CLK, CLR);

    always begin
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("Exercise9_btb.vcd");
        $dumpvars(0, Exercise9_tb);

        CLK = 0;
        CLR = 1;
        #10 CLR = 0;

        #200 $finish;
    end

    initial begin
        $monitor("Time = %0t, A = %b, B = %b", $time, A, B);
    end

endmodule
```

Figure 9: Testbench

We initially set the clock to be 0, and then set the CLR to 1, so that the circuit is reset to the 0 condition, and then after a delay set it to 0, to start the counting, with the stop time of 200ns.

The waveforms thus obtained in the GTKWave file are provided below:

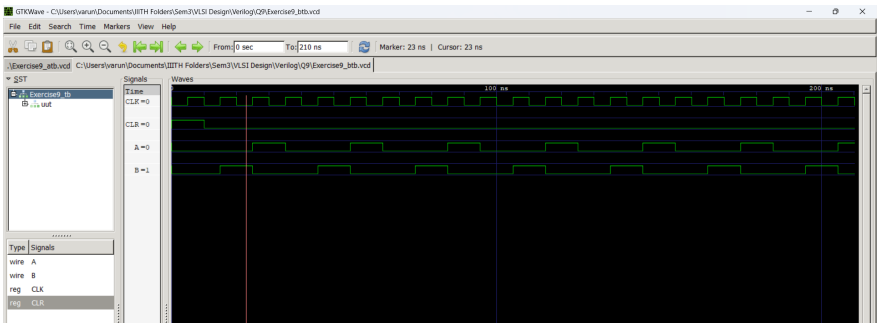


Figure 10: Waveforms in GTKWave

Thus with the arrival of every rising edge of the clock, since we start from {A, B} = {0,0}, the outputs can be seen to be in the sequence {00, 01, 10, 00, 00 . . . } where the first bit represents A,

and the latter representing B.

Exercise 10 Finite State Machines

In this exercise, we implement the Moore and Mealy Finite State Machines.
In the Moore's FSM Model, the output is **only the function of the present state, and not the inputs**. Whereas, in the Mealy's FSM Model, the output is **the function of both the inputs and the present state.**

Simplified Block Diagrams for the Mealy and Moore Machines are provided below:

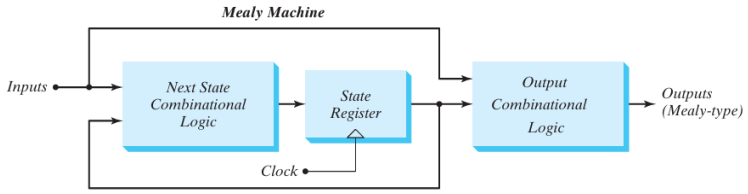


Figure 11: Mealy Machine

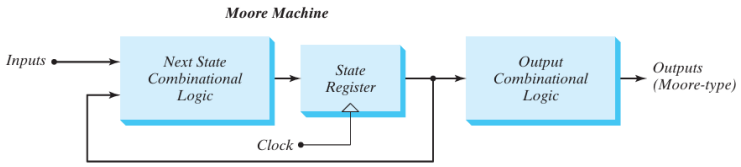


Figure 12: Moore Machine

n a Moore model, the outputs of a sequential circuit are synchronized with the clock since they depend solely on the flip-flop outputs, which are clocked. In contrast, a Mealy model's outputs can change during a clock cycle if the inputs change, and may produce temporary incorrect values due to delays between input changes and flip-flop updates. To synchronize a Mealy circuit, the inputs must align with the clock, and outputs should be sampled right before the clock edge. Inputs are adjusted at the inactive clock edge to ensure stability before the active edge. Thus, the Mealy machine's output is taken just before the active clock edge.
Naturally, we expect the state-transition diagrams and the state table representations to be different for the 2 machines. In the state-table, the output column would depend on the input, in the case of a Mealy machine. And, in the state-transition diagram, we represent the output too with the input, in the transition arrows, to indicate that the output is indeed dependent on the input in the case of a Mealy Machine.

For this entire exercise, I have written the behavioural verilog HDL.

a) Moore FSM

We are asked to implement the Moore FSM for the FSM Diagram given below:

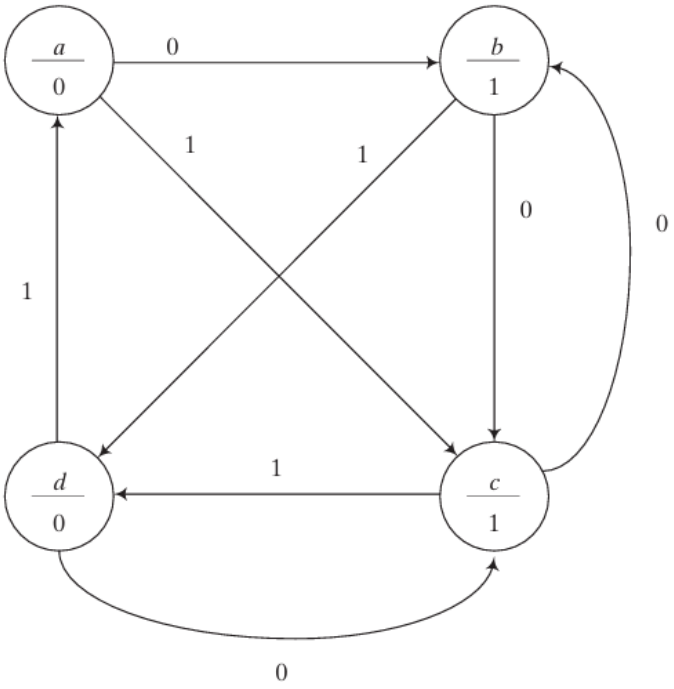


Figure 13: Given Moore FSM Diagram

The state-table that can be thus derived, is given below:

Current State	**Next State**		**Output**
	I/P = 0	**I/P = 1**	
a	b	c	0
b	c	d	1
c	b	d	1
d	c	a	0

The verilog HDL for this state-flow is given below:



```
// Q.10) from the Verilog Practice Sheet, Moore model FSM
// state model behaviour

module q10_moore(Y, X, CLK, RST);
    input X, CLK, RST;
    output reg Y;

    parameter STATE_A = 2'b00, STATE_B = 2'b01, STATE_C = 2'b10, STATE_D = 2'b11;
    reg [1:0] curr_state, next_state;

    always @(posedge CLK or posedge RST) begin
        if (RST)
            curr_state <= STATE_A;
        else
            curr_state <= next_state;
    end

    always @(*) begin
        case (curr_state)
            STATE_A: begin next_state = X ? STATE_C : STATE_B; Y = 1'b0; end
            STATE_B: begin next_state = X ? STATE_D : STATE_C; Y = 1'b1; end
            STATE_C: begin next_state = X ? STATE_D : STATE_B; Y = 1'b1; end
            STATE_D: begin next_state = X ? STATE_A : STATE_C; Y = 1'b0; end
            default: next_state = STATE_A;
        endcase
    end

endmodule
```

Figure 14: Module for Moore Machine

The implementation is straight-forward, with the states A to D being designated binary strings 00, 01, 10 and 11 respectively. Two wires `curr_state` and `next_state` are maintained to keep track of the current, and forthcoming state. The clock and reset are set to be positive edge triggering, and in the `always` block where we are assigning the `next_state`, we also assign the output value `Y`, according to the state table above.

The testbench for the module file is given below:

```
// Testbench for q10 Moore model
`timescale 1ns / 1ns
`include "q10_moore.v"

module q10_moore_tb;
    reg X, CLK, RST;
    wire Y;

    q10_moore uut(Y, X, CLK, RST);

    always begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("q10_moore_tb.vcd");
        $dumpvars(0, q10_moore_tb);

        X = 0;
        RST = 1;
        force uut.curr_state = 2'b00;
        #10 RST = 0;
        release uut.curr_state;


        #10 X = 1;
        #10 X = 0;
        #10 X = 1;
        #10 X = 1;
        #10 X = 0;
        #10 X = 0;
        #10 X = 1;
        #10 X = 0;

        $finish;
    end

    always @(posedge CLK) begin
        $display("X=%b, Y=%b", X, Y);
    end
endmodule
```

Figure 15: Testbench for Moore Machine

As can be seen in the testbench snippet above, we force the initial state to be 00, without loss of generality, (say we choose state ‘a’ as the default state) while resetting the circuit.

Upon execution of this file, the GTKWave plots obtained are given below:

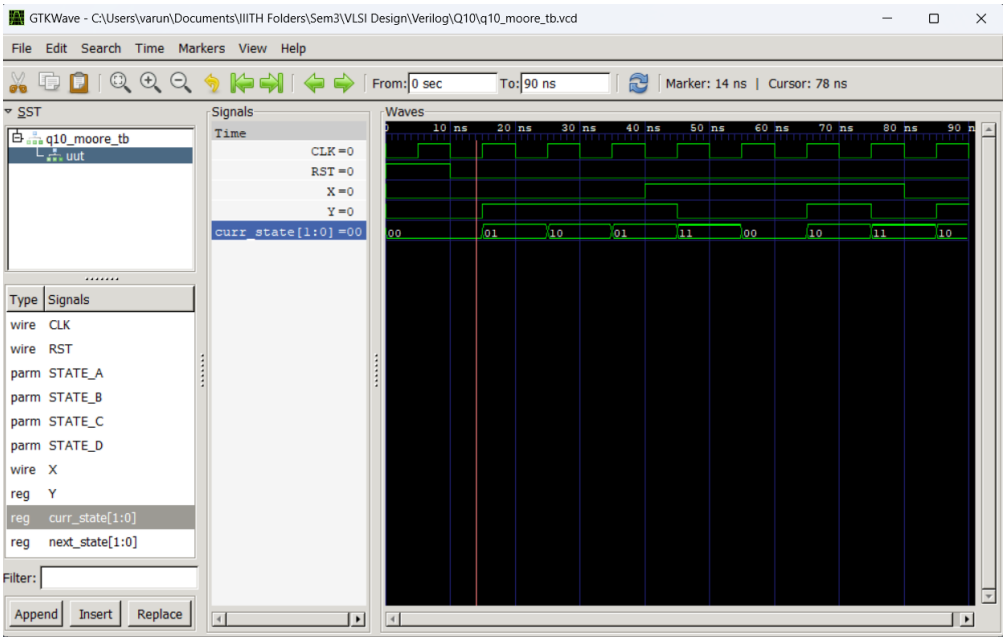


Figure 16: GTKWave plots for the given Moore Machine

Let's try to verify this, at the red cursor line. Here we can see that we are currently in state A (00), with the input at the next rising edge of the clock (X) being 0. Thus we will move to state B (01), with the output being 1 (output 1 corresponds to state B in the state diagram/table). Next active edge arrival, the input X is still 0, and we thus move to state C, with the output remaining 1. Now the input X is 1, and the next clock arrival we move to state D, with the output Y becoming 0.

b) Mealy FSM

We now implement a Mealy FSM from the given state-transition diagram below:

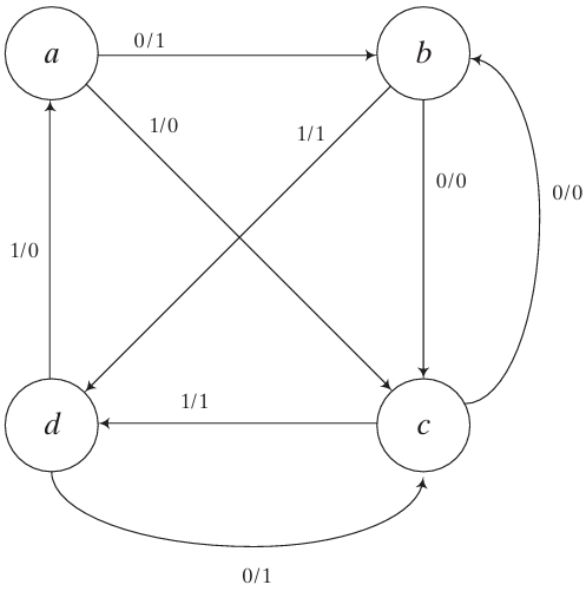
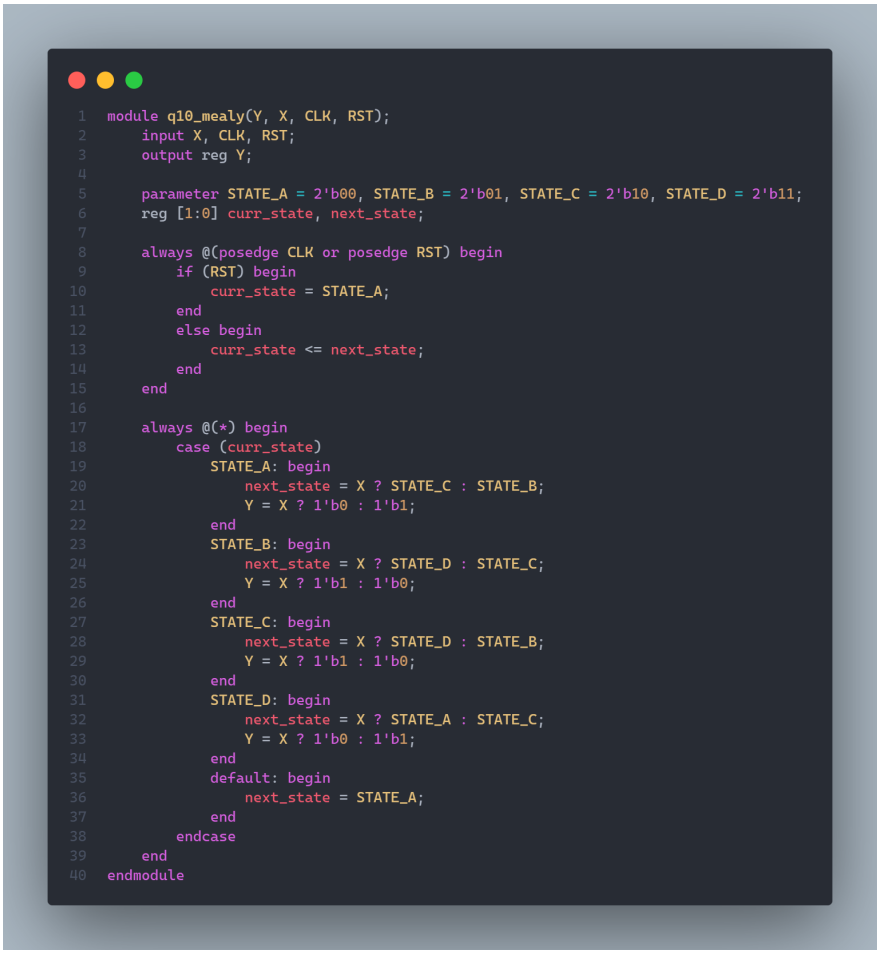


Figure 17: Mealy FSM Diagram

The state-table that can be thus derived is given below:

Current State	**Next State**		**Output**	
	I/P = 0	**I/P = 1**	**I/P = 0**	**I/P = 1**
a	b	c	1	0
b	c	d	0	1
c	b	d	0	1
d	c	a	1	0

The verilog HDL for this state-transition diagram is given below:



```
module q10_mealy(Y, X, CLK, RST);
    input X, CLK, RST;
    output reg Y;

    parameter STATE_A = 2'b00, STATE_B = 2'b01, STATE_C = 2'b10, STATE_D = 2'b11;
    reg [1:0] curr_state, next_state;

    always @(posedge CLK or posedge RST) begin
        if (RST) begin
            curr_state = STATE_A;
        end
        else begin
            curr_state <= next_state;
        end
    end

    always @(*) begin
        case (curr_state)
            STATE_A: begin
                next_state = X ? STATE_C : STATE_B;
                Y = X ? 1'b0 : 1'b1;
            end
            STATE_B: begin
                next_state = X ? STATE_D : STATE_C;
                Y = X ? 1'b1 : 1'b0;
            end
            STATE_C: begin
                next_state = X ? STATE_D : STATE_B;
                Y = X ? 1'b1 : 1'b0;
            end
            STATE_D: begin
                next_state = X ? STATE_A : STATE_C;
                Y = X ? 1'b0 : 1'b1;
            end
            default: begin
                next_state = STATE_A;
            end
        endcase
    end
endmodule
```

Figure 18: Mealy FSM Module

Here we notice the difference in implementation between the Mealy and Moore FSMs. When we are changing the `curr_state`, notice that in this module, we are also assigning the output Y based on the input X's value, which wasn't the case in the Moore FSM module. Again, here we assume the default state to be `STATE_A` (00)

The testbench for the module is given below:

```verilog
// Module for Q10) Mealy machine testbench
`timescale 1ns/1ps
`include "q10_mealy.v"
module q10_mealy_tb;
    reg X, CLK, RST;
    wire Y;

    q10_mealy uut(Y, X, CLK, RST);

    always begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("q10_mealy_tb.vcd");
        $dumpvars(0, q10_mealy_tb);

        X = 0;
        RST = 1;
        force uut.curr_state = 2'b00;
        #10 RST = 0;
        release uut.curr_state;

        #10; X = 0;
        #10; X = 1;
        #10; X = 1;
        #10; X = 0;
        #10; X = 0;
        #10; X = 1;
        #10; X = 0;
        #10; X = 0;

        $finish;
    end

    initial begin
        $monitor("Time = %0t, RST = %b, X = %b, Y = %b, State = %b", $time, RST, X, Y, uut.curr_state);
    end
endmodule
```

Figure 19: Mealy FSM Testbench

Again here, we force an initial condition during reset, and then observe the waveforms over different input (X) values. The GTKWave plots obtained are given below.

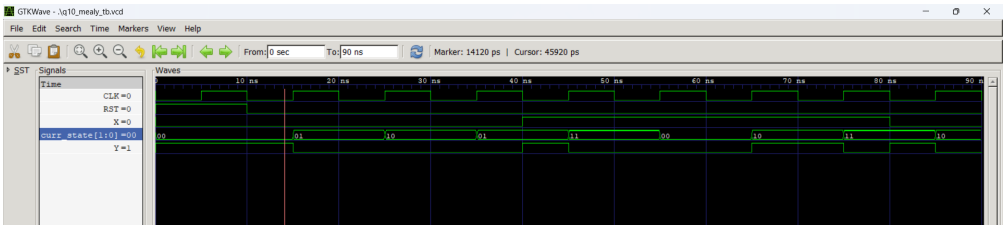


Figure 20: GTKWave waveforms for the Mealy machine

Initially, since the input is 0, and the state forced is A (00), from the state table we see that the output Y = 1, which is also the case in the GTKWave waveform. On the arrival of the next clock pulse, we transition to state B (01), and output becomes 0 because the input continues to be 0, and so on, the waveforms can be traced from the state table given above.

Exercise 11 Two 1s and 0s detector

In this exercise, we implement a machine that detects two 1s and 0s in any order. The inputs are as follows:

1. **CLK** : used to clock the FSM

2. **RESET** : asserted high to reset the FSM to its initial state

3. **ONE** : asserted high to input a "1". Note that this signal may stay high for many cycles (eg, it's generated by a button press) before returning low. Each high period should count as a single "1" input, ie, to inputs two "1"s in series, the signal must return low in between the first and second "1".

4. **ZERO** : asserted high to input a "0". This signal has the same timing protcol as ONE above.

And, the output **OUT** is asserted high when at least two "0"s and two "1"s have occurred as inputs.

Also, I've taken an implicit assumption that ZERO and ONE both won't be asserted HIGH at the same time, atleast as recognised by the CLK. Also, we can easily identify that the output depends solely on the present state, and thus is a Moore Machine.

We can identify nine distinct states, as given below:

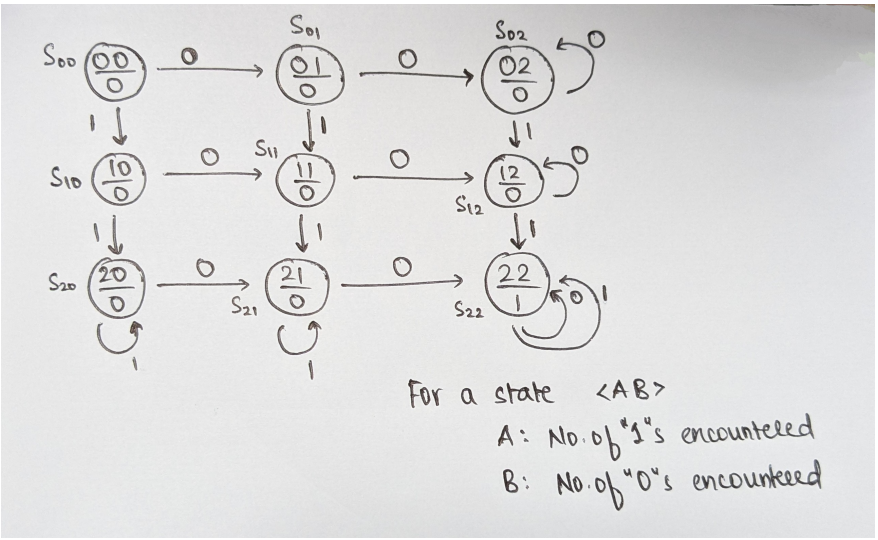
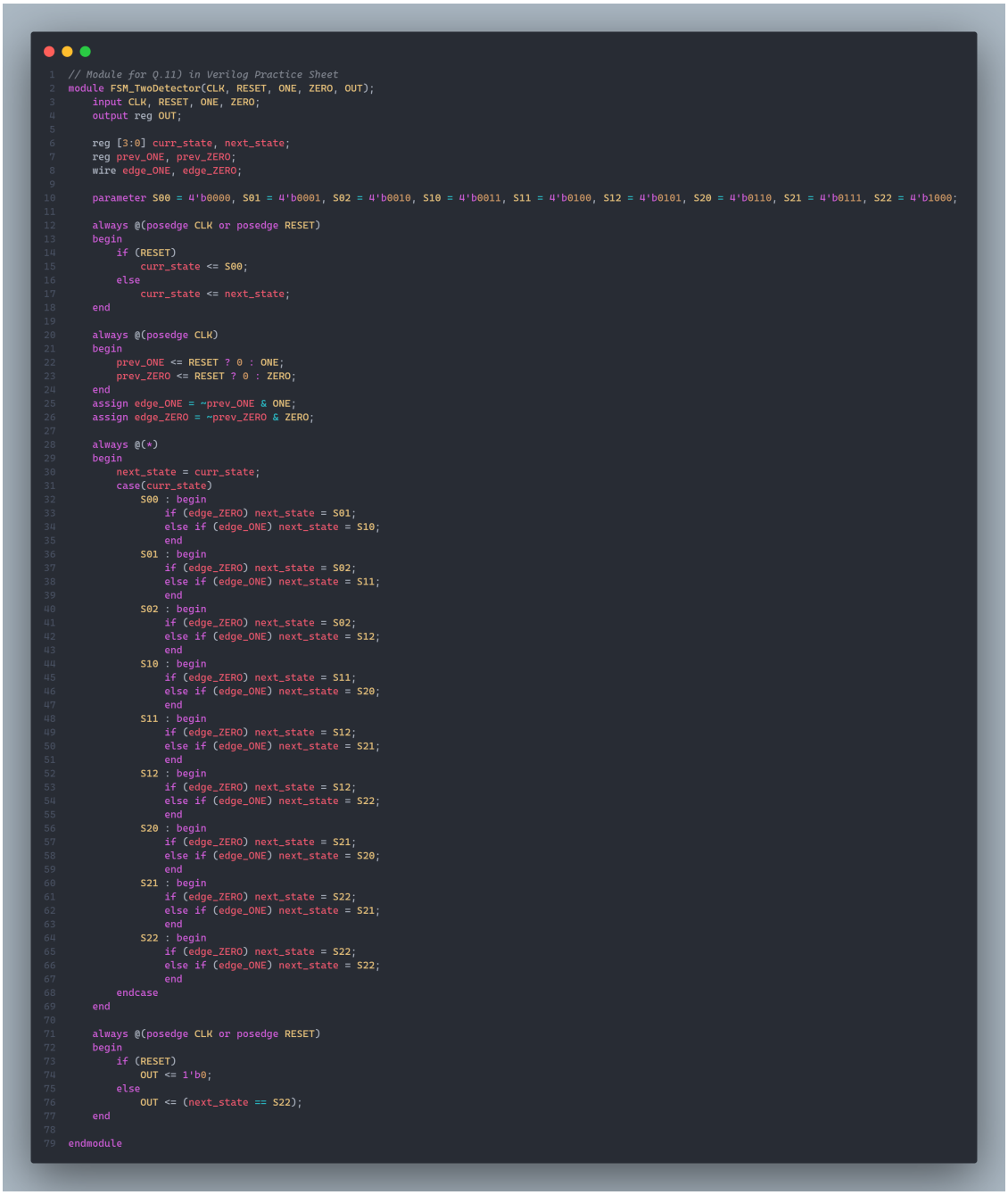


Figure 21: State Transition Diagram

Thus the state-table can be deduced, given below:

Current State	**Next State**		**Output**	
	I/P = 0	**I/P = 1**	**I/P = 0**	**I/P = 1**
a	b	c	1	0
b	c	d	0	1
c	b	d	0	1
d	c	a	1	0

The verilog module is given below:



```verilog
// Module for Q.11) in Verilog Practice Sheet
module FSM_TwoDetector(CLK, RESET, ONE, ZERO, OUT);
    input CLK, RESET, ONE, ZERO;
    output reg OUT;

    reg [3:0] curr_state, next_state;
    reg prev_ONE, prev_ZERO;
    wire edge_ONE, edge_ZERO;

    parameter S00 = 4'b0000, S01 = 4'b0001, S02 = 4'b0010, S10 = 4'b0011, S11 = 4'b0100, S12 = 4'b0101, S20 = 4'b0110, S21 = 4'b0111, S22 = 4'b1000;

    always @(posedge CLK or posedge RESET)
    begin
        if (RESET)
            curr_state <= S00;
        else
            curr_state <= next_state;
    end

    always @(posedge CLK)
    begin
        prev_ONE <= RESET ? 0 : ONE;
        prev_ZERO <= RESET ? 0 : ZERO;
    end
    assign edge_ONE = ~prev_ONE & ONE;
    assign edge_ZERO = ~prev_ZERO & ZERO;

    always @(*)
    begin
        next_state = curr_state;
        case(curr_state)
            S00 : begin
                if (edge_ZERO) next_state = S01;
                else if (edge_ONE) next_state = S10;
                end
            S01 : begin
                if (edge_ZERO) next_state = S02;
                else if (edge_ONE) next_state = S11;
                end
            S02 : begin
                if (edge_ZERO) next_state = S02;
                else if (edge_ONE) next_state = S12;
                end
            S10 : begin
                if (edge_ZERO) next_state = S11;
                else if (edge_ONE) next_state = S20;
                end
            S11 : begin
                if (edge_ZERO) next_state = S12;
                else if (edge_ONE) next_state = S21;
                end
            S12 : begin
                if (edge_ZERO) next_state = S12;
                else if (edge_ONE) next_state = S22;
                end
            S20 : begin
                if (edge_ZERO) next_state = S21;
                else if (edge_ONE) next_state = S20;
                end
            S21 : begin
                if (edge_ZERO) next_state = S22;
                else if (edge_ONE) next_state = S21;
                end
            S22 : begin
                if (edge_ZERO) next_state = S22;
                else if (edge_ONE) next_state = S22;
                end
        endcase
    end

    always @(posedge CLK or posedge RESET)
    begin
        if (RESET)
            OUT <= 1'b0;
        else
            OUT <= (next_state == S22);
    end

endmodule
```

Figure 22: Verilog Module

The states are designated as above in the state diagram. The variables `prev_ONE` and `prev_ZERO` are declared which will help us use the rising edge of the ONE or ZERO input to update the `next_state`. Say for instance, at every rising clock edge, `prev_ONE` stores the value of ONE if the RESET input/mode isn't active. So if input ONE is HIGH over 2 clock cycles, the parameter `prev_ONE` will be HIGH at the first pulse, and at the second pulse, it would not be considered as a second HIGH, because `edge_ONE` would be 0 for the 2nd pulse.
With this implementation, we convert our inputs from `ONE` or `ZERO` which are level-triggered, in a way, to `edge_ZERO` and `edge_ONE` which are edge-triggered, and in this case, positive edge triggered.

And hence, in the end we assign the OUT to be 1, when the `next_state` becomes the same as `S22`, as asked in the question.

The testbench for this module is given below:

```v
// Testbench for FSM_TwoDetector.v
`timescale 1ns/1ps
`include "FSM_TwoDetector.v"

module FSM_TwoDetector_tb;
    reg CLK, RESET, ONE, ZERO;
    wire OUT;

    always begin
       forever #5 CLK = ~CLK;
    end
    FSM_TwoDetector uut(CLK, RESET, ONE, ZERO, OUT);


    initial begin
        $dumpfile("FSM_TwoDetector_tb.vcd");
        $dumpvars(0, FSM_TwoDetector_tb);

        CLK = 0; RESET = 1; ONE = 0; ZERO = 0;
        #10 RESET = 0;
        #10 ONE = 1;
        #10 ONE = 1;
        #10 ONE = 1;
        #10 ONE = 0;
        #10 ZERO = 1;
        #10 ZERO = 1;
        #10 ZERO = 1;
        #10 ZERO = 1;
        #10 ZERO = 0;
        #10 ONE = 1;
        #10 ONE = 0;
        #10 ZERO = 1;
        #10 ZERO = 0;
        #10 ONE = 1;
        #10 ONE = 0;
        #10 ZERO = 1;
        #10 ZERO = 0;
        #10 ONE = 1;
        #10 ONE = 0;

        // Case 2
        #10 RESET = 1;
        #10 RESET = 0;
        #10 ONE = 1;
        #10 ONE = 0;
        #10 ZERO = 1;
        #10 ZERO = 0;
        #10 ONE = 1;
        #10 ONE = 0;
        #10 ZERO = 1;
        #10 ZERO = 0;
        #10 $finish;
    end
endmodule
```

Figure 23: Testbench

The testbench is pretty straight-forward. We mainly just vary the inputs ONE and ZERO over different values, and take up 2 examples of these. The GTKWave plots obtained are given below:

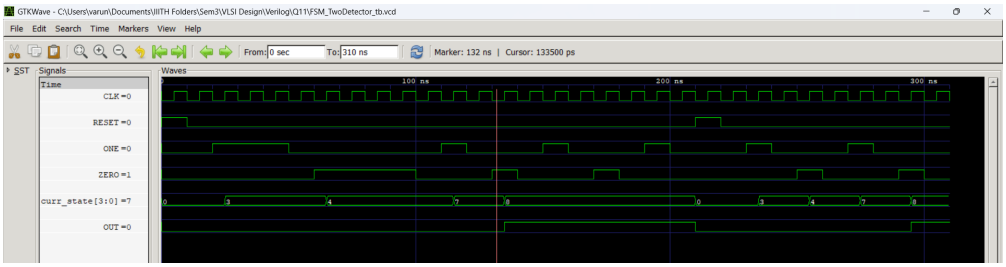


Figure 24: GTKWave Plots

Let's analyze this from the start for the first example. After the reset, the first change is in input ONE, which is recognised by the clock, and the state thus moves to 3 (i.e. 0011), post which a ZERO is sensed, and then we move to state 4 (0100). The next time the clock senses a ONE at 115ns and thus the state transitions to state 7 (0111), post which it senses a ZERO, and thus moves to state 8, which is where the OUT becomes 1, as the condition in the module is met.